



UNIVERSIDAD SERGIO ARBOLEDA

**COLONIA DE HORMIGAS APLICADA AL PROBLEMA
DEL VIAJANTE (TSP): INSTANCIAS DE 20, 2 000 Y 200 000
CIUDADES**

Inteligencia Artificial

Mario Jiménez

Juan David Andrade

Camilo Andrés Díaz García

*Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Introducción	2
2. Descripción del problema y espacio de búsqueda factorial	2
3. Diseño de la solución	3
3.1. Elección de lenguaje	3
3.2. Construcción del tour	4
3.3. Número de agentes	4
3.4. Estructuras de datos y estrategias de ahorro	6
4. Análisis de complejidad	7
5. Metodología experimental	8
6. Resultados por instancia	9
6.1. $n = 20$	9
6.2. $n = 2\,000$	10
6.3. $n = 200\,000$	12
7. Análisis de tiempo y memoria	13
7.1. Memoria: densa contra dispersa	13
7.2. Memoria por componente	14
7.3. Efecto de K y de la segunda feromona	15
7.4. Varianza entre semillas por parámetro	16
7.5. Longitud contra referencias	16
7.6. Costo de m y escalamiento con n	17
7.7. Igual presupuesto de cómputo en $n = 200\,000$	18
7.8. Tours finales	18
8. Limitaciones	19
9. Conclusiones	19

1 Introducción

El problema del viajante (*Traveling Salesman Problem*, TSP) consiste en encontrar el recorrido más corto que visita una sola vez cada ciudad de un conjunto y regresa al punto de partida. Es uno de los problemas de optimización combinatoria más estudiados porque, siendo fácil de enunciar, su espacio de soluciones crece de forma factorial con el número de ciudades: para instancias grandes ninguna computadora actual puede revisar todos los recorridos posibles, y hace falta una estrategia distinta a la fuerza bruta.

Este trabajo aplica la optimización por colonia de hormigas (*Ant Colony Optimization*, ACO) al TSP en tres instancias de tamaño muy distinto: 20, 2 000 y 200 000 ciudades, midiendo el desempeño en tiempo y en memoria, mostrando cómo ahorrar ambos recursos y sustentando cada decisión con una medición real, no con una suposición. Con 200 000 ciudades, una matriz de distancias o de feromona completa ($n \times n$) ocuparía 320 GB (ver sección 4), muy por encima de la memoria de cualquier computador personal; ese es el motivo central por el que este proyecto gira en gran medida alrededor de **cómo representar el problema sin pagar ese costo**, y no solo alrededor de encontrar tours cortos.

2 Descripción del problema y espacio de búsqueda factorial

En un TSP simétrico ($d_{ij} = d_{ji}$) con origen fijo (siempre se empieza en la misma ciudad, sin pérdida de generalidad porque el tour es un ciclo), el número de recorridos distintos es

$$\frac{(n-1)!}{2}, \quad (1)$$

porque hay $(n-1)!$ formas de ordenar las $n-1$ ciudades restantes, y cada tour se cuenta dos veces (una por cada sentido de recorrido). Esta cantidad crece más rápido que cualquier exponencial en n : para $n = 20$ ya es del orden de 10^{16} (más recorridos que segundos han pasado desde el big bang, multiplicados varias veces); para $n = 2000$ y $n = 200000$ el número de dígitos se cuenta en miles y en cientos de miles, respectivamente (ver figura de crecimiento factorial, calculada con la función `lgamma` para evitar desbordar cualquier tipo numérico). Esta es la razón de fondo del ejercicio: ningún algoritmo de fuerza bruta, por rápido que sea el computador, revisa ese espacio en un tiempo razonable, así que hace falta una metaheurística que explore una fracción pequeña del espacio y aun así encuentre soluciones buenas.

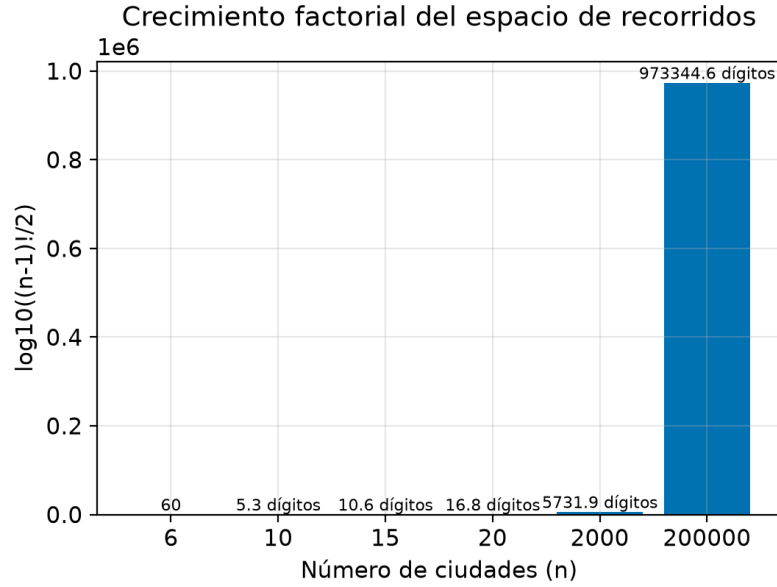


Figura 1: Crecimiento de $\log_{10}((n-1)!/2)$ para $n = 6, 10, 15, 20, 2000, 200000$.

Tabla 1: Recorridos posibles $(n-1)!/2$, calculado con lgamma.

n	$(n-1)!/2$	$\log_{10}$
6	6.000e+01	1.78
10	1.814e+05	5.26
15	4.359e+10	10.64
20	6.082e+16	16.78
2000	$> 10^{5732}$ (5732 dígitos)	5731.92
200000	$> 10^{973345}$ (973345 dígitos)	973344.55

Held-Karp resuelve el TSP de forma exacta con programación dinámica en $O(n^2 2^n)$, muchísimo mejor que $O(n!)$ pero todavía exponencial: en este trabajo solo es viable para $n = 20$ (ver sección 4), y se usa exclusivamente para verificar qué tan cerca del óptimo llega ACO en la instancia más pequeña. Para $n = 2000$ y $n = 200000$ no existe una forma computable de obtener el óptimo exacto a esa escala, así que la calidad solo se puede comparar contra referencias, no contra el valor óptimo real (ver sección 7 para el detalle de esas referencias).

3 Diseño de la solución

3.1 Elección de lenguaje

Se implementó en C++20 por su manejo directo de memoria, su soporte de estructuras de datos eficientes (vectores contiguos, tipos atómicos) en la librería estándar, y porque permite paralelismo simple con OpenMP: una directiva por bucle basta para repartir la

construcción de tours entre hilos, sin reescribir la lógica secuencial del algoritmo. Ese paralelismo con bajo costo de código adicional, junto con partir de un código base ya validado, fue lo que inclinó la decisión hacia C++ y OpenMP para este proyecto.

3.2 Construcción del tour

Esta subsección describe la parte del código (`buildTour` en `src/aco_tsp.cpp`) que cada hormiga ejecuta para armar un tour, y las fórmulas exactas que implementa. El TSP se representa como un grafo completo de n ciudades, con una distancia d_{ij} entre cada par (i, j) (euclidiana, en este trabajo). Cada arista (i, j) tiene, además de su distancia, un valor de feromona τ_{ij} que el algoritmo actualiza en cada iteración; τ_{ij} alto significa “muchas hormigas buenas pasaron por aquí en el pasado reciente”.

En cada paso de construcción, una hormiga que está en la ciudad i elige la siguiente ciudad j , entre las no visitadas, con probabilidad

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in N_i^k} \tau_{il}^\alpha \eta_{il}^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}}, \quad (2)$$

donde N_i^k es el conjunto de ciudades que la hormiga k aún no ha visitado (o, en la versión dispersa de este trabajo, la lista de K vecinas más cercanas de i que aún no se visitaron, ver sección 3.4), y η_{ij} es la **heurística**: entre más cerca esté una ciudad, más atractiva es, con independencia de la feromona. Los parámetros α y β controlan cuánto pesa la feromona frente a la heurística: con $\alpha = 0$ solo importa la distancia (un vecino más cercano estocástico); con $\beta = 0$ solo importa la feromona acumulada, lo que típicamente lleva a estancamiento (todas las hormigas convergen al mismo tour, así no sea bueno). La ecuación (2) no elige siempre la arista de mayor peso: el código la implementa como una **ruleta** probabilística sobre los pesos de las candidatas, lo que permite explorar en vez de repetir siempre el mismo camino y escapar de óptimos locales, la razón de fondo de por qué dos corridas con distinta semilla (misma configuración) dan tours distintos, como se ve en las curvas de convergencia de la sección 6.

3.3 Número de agentes

Se adopta $m = \min(n, 20000)$: $n = 20$ con $m = 20$; $n = 2000$ con $m = 2000$; $n = 200000$ con $m = 20000$ (las 200000 ciudades se ejecutan siempre completas). La convención de la literatura del Ant System sugiere $m = n$, lo que para $n = 200000$ resulta impracticable en esta máquina; el tope de 20 000 es una decisión de ingeniería, no un óptimo teórico.

La comparación a igual presupuesto de cómputo en $n = 200000$ ($m = 2048$ contra

$m = 20000$, 15 min de reloj por semilla, 3 semillas; resultados/resultado_n200000_punto5.csv y resultados/resultado_n200000_punto6.csv) es el dato central de esta subsección; se agrega además una tercera configuración deliberadamente barata ($m = 2048$, solo 5 iteraciones, mismas 3 semillas; resultados/resultado_n200000_punto7.csv) para mostrar el otro extremo del compromiso:

Tabla 2: Tres configuraciones en $n=200\,000$: dos a igual presupuesto de tiempo ($m=20\,000$, 10 iteraciones; $m=2\,048$, 108 iteraciones) y una configuración barata de contraste ($m=2\,048$, 5 iteraciones).

Configuración	Iteraciones	Tiempo total (s)	L_{mejor} media	Desv. estándar	Mejora vs. NN	Memoria pico (MB)
$m = 20000$	10	838,3	387,308	0,190	1,28 %	102,4
$m = 2048$	≈ 108	904,8	385,485	0,550	1,75 %	102,4
$m = 2048$ (barata)	5	44,1	390,486	0,518	0,47 %	102,5

Se prefirió, en la práctica, $m = 2048$ sobre $m = 20000$: a igual tiempo de reloj dio mejor calidad en las 3 de 3 semillas probadas. El tope de 20 000 se sostiene entonces por la convención de la literatura y el presupuesto de tiempo *total* del proyecto, no por una ventaja de calidad medida.

La razón detrás de esto es de tiempo y paralelismo, no de memoria: en esta implementación cada hilo reutiliza un único buffer de estado (sección 3.4), así que el pico de memoria medido no depende de cuántas hormigas se ejecuten por iteración, y de hecho ambas configuraciones midieron prácticamente el mismo pico (102,4 MB). Lo que sí cambia es cómo se reparte el tiempo de cómputo: con OpenMP el paralelismo tiene un límite práctico cercano al número de núcleos disponibles, así que agregar hormigas más allá de ese límite añade hilos que compiten por los mismos núcleos y por el mismo ancho de banda de memoria, sin acelerar el trabajo de forma proporcional. Invertir el mismo presupuesto de tiempo en más iteraciones con una colonia moderada, en cambio, aprovecha ese tiempo íntegro para refinar la feromona paso a paso en vez de gastarlo en coordinar más hormigas por vuelta, lo que es consistente con que $m = 2048$ haya dado mejor calidad que $m = 20000$ a igual tiempo de reloj y con memoria equivalente.

La tercera fila de la Tabla 2 muestra el otro extremo de este mismo compromiso: recortar las iteraciones a solo 5 ahorra tiempo de forma proporcional (44,1 s frente a los 838–905 s de las otras dos configuraciones), pero no memoria (el pico sigue en ≈ 102 MB, por la misma razón de arriba), y la calidad cae de forma notoria (0,47 % de mejora sobre el vecino más cercano, frente a 1,28 % y 1,75 %). Hay entonces un límite práctico a cuánto se puede recortar antes de que la solución deje de ser útil: el ahorro de recursos de este trabajo tiene sentido mientras siga habiendo suficientes iteraciones para que la feromona alcance a guiar la búsqueda.

3.4 Estructuras de datos y estrategias de ahorro

La razón central del enunciado (“mostrar cómo ahorrar recursos”) se resuelve en la representación de datos, no en un truco aislado. La Tabla 3 resume las nueve estrategias implementadas, cada una con su motivo, su costo antes y después, y la medición que la respalda (detalle completo en docs/estrategias_ahorro.md). La idea que las conecta a todas es evitar cualquier estructura de tamaño $O(n^2)$: en vez de una matriz completa de distancias y feromonas, cada ciudad solo conoce sus K vecinas más cercanas (encontradas por una rejilla espacial en $O(nK)$, no por comparación exhaustiva en $O(n^2)$; esta técnica corresponde a lo que en la literatura y en implementaciones comparables se denomina *estructura jerárquica espacial* o *hier*, una poda por vecindad que evita recorrer todas las ciudades), y esas ciudades se renumeran por celda para que la memoria consultada por una hormiga esté físicamente cerca en el arreglo (localidad de caché). Ninguna hormiga guarda su tour completo: cada hilo reutiliza un único buffer y deposita feromona con operaciones atómicas al terminar, así que la memoria no depende de m .

Esa actualización de feromona ocurre una sola vez por iteración (relevante para la sección 3.3 sobre el costo de m), en dos pasos: evaporación,

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad (3)$$

que “olvida” decisiones antiguas y evita que la feromona crezca sin límite; y depósito,

$$\Delta \tau_{ij}^k = \frac{Q}{L_k} \quad \text{si la hormiga } k \text{ usó la arista } (i, j), \quad \tau_{ij} \leftarrow \tau_{ij} + \sum_k \Delta \tau_{ij}^k, \quad (4)$$

proporcional a la calidad del tour (L_k : su longitud), con $Q = q_{fac} \cdot L_{nn}/m$ normalizado para ser comparable entre instancias de tamaño distinto. Este trabajo agrega, de forma opcional, una **segunda capa de feromona** τ_2 , reforzada solo por el mejor tour global, con exponente $\alpha_2 < \alpha$; el peso final de cada arista es $w_{ij} = \tau_{ij}^\alpha \tau_{2,ij}^{\alpha_2} \eta_{ij}^\beta$, el “peso precalculado por iteración” de la Tabla 3.

Tabla 3: Estrategias de ahorro de tiempo y memoria (detalle en docs/estrategias_ahorro.md).

Estrategia	Recurso	Antes	Después	Medición
Sin matriz $n \times n$; lista de K vecinas	Memoria	$O(n^2)$	$O(nK)$	320 GB (densa) contra 96,8 MB medidos (dispersa), $n = 200\,000$
Vecinas por rejilla espacial	Tiempo de preparación	$O(n^2)$	$O(nK)$	Preparación: 0,149 s, $n = 200\,000$
Renumeración por celda	Tiempo (caché)	Acceso disperso	Acceso contiguo	Decisión de diseño, sin comparación aislada
Hormigas sin guardar su tour; buffer por hilo	Memoria	$O(mn)$	$O(n)$ por hilo	~ 16 GB evitados con $m = 20\,000$, $n = 200\,000$
Pesos precalculados por iteración	Tiempo	Recalcular por paso	Una vez por iteración	Cada paso: suma y ruleta de K términos
η^β precalculado	Tiempo	Recalcular por iteración	Una vez al inicio	La distancia no cambia
Paralelismo por hormiga (OpenMP)	Tiempo	Secuencial	Hasta 12 hilos	Ver sección 7
float y arreglos contiguos	Memoria y caché	double	float	Verificación en double coincide en todas las corridas
Segunda feromona solo en aristas candidatas	Memoria	$O(n^2)$ si fuera densa	$O(nK)$	Mismo argumento que la primera fila

4 Análisis de complejidad

Cada afirmación de costo de este trabajo tiene una derivación teórica y una comprobación empírica (sección 7); aquí se presenta solo la derivación.

Tabla 4: Complejidad en tiempo, derivada por componente.

Componente	Complejidad	Por qué
Espacio de búsqueda	$(n-1)!/2$; Held-Karp $O(n^2 2^n)$	Fuerza bruta inviable para cualquier n práctico; Held-Karp es exacto pero exponencial, viable solo en $n = 20$ (nota 1); ACO explora una fracción minúscula del espacio en tiempo polinomial, sin garantía de optimalidad
Construcción de una hormiga	$O(nK)$ (+ reserva)	K candidatas por paso; la búsqueda de reserva por rejilla (cuando las K ya están visitadas) es más cara al final del tour, con pocas ciudades libres
Una iteración	$O(mnK)$; $O(n^2 K)$ si $m = n$	m hormigas $\times O(nK)$ cada una, más $O(nK)$ de evaporación y recálculo de pesos; con matriz densa serían $O(mn^2)$ en tiempo y $O(n^2)$ en memoria (comparación aco_tsp/aco_denso en la sección 7)

Nota 1: $2^{20} \approx 10^6$, así que $n^2 2^n \approx 4 \times 10^8$ operaciones es viable en segundos; para $n = 2\,000$ o $n = 200\,000$, 2^n desborda cualquier computador existente.

Tabla 5: Complejidad en memoria, derivada por estructura.

Estructura	Tamaño	Nota
Vecinas, distancias, feromona(s) y pesos	$O(nK)$	Por instancia, no por hormiga
Estado de construcción (buffers de tour y de visitado)	$O(n)$ por hilo	Independiente de m : las hormigas no guardan su tour, así que más hormigas por iteración no agrega arreglos, solo más trabajo de CPU repitiendo el mismo buffer

Paralelismo. Las hormigas se construyen en paralelo casi sin coordinación entre hilos; la fracción estrictamente secuencial s de cada iteración (evaporación, pesos, mejor global) acota, por la ley de Amdahl, la aceleración máxima alcanzable con p hilos:

$$\text{Aceleración máxima} = \frac{1}{s + \frac{1-s}{p}}. \quad (5)$$

5 Metodología experimental

Máquina. Intel Core i7-8750H, 6 núcleos físicos / 12 hilos lógicos, 15,88 GB de RAM, Windows 11 Home (docs/entorno.md). Todos los tiempos de este informe se refieren a esta máquina.

Instancias y semillas. Puntos generados de forma uniforme en el cuadrado unidad, a partir de una semilla entera. El barrido de parámetros usa 5 semillas por valor en $n = 2000$; la selección final de configuración y el resultado oficial de $n = 2000$ usan 8 semillas; $n = 20$ usa 5 semillas contra Held-Karp; $n = 200000$ usa hasta 3 semillas por el costo de cómputo (sección 3.3).

Parámetros. Configuración de partida: $K = 8$, $\alpha = 1.5$, $\beta = 5$, $\rho = 0.1$, $q_{fac} = 3$ (tomada de una nota preliminar del enunciado). Se probaron distintos valores de cada parámetro por separado y distintas combinaciones (resultados/barrido_parametros.csv), y se seleccionó la configuración con mejor resultado promedio: activar la segunda capa de feromona ($\text{two} = 1$, $\alpha_2 = 1.0$) sobre la configuración de partida. Detalle completo en docs/decisiones_diseno.md.

Criterios de parada. El algoritmo se detiene por el primero de tres criterios que se cumpla: un número máximo de iteraciones, un presupuesto de tiempo total (`--time`), o un número de iteraciones consecutivas sin mejorar el mejor global (*stall*). Los tres se compararon en $n = 2000$, 5 semillas cada uno (resultados/barrido_criterio_parada.csv): con 50 iteraciones fijas la longitud media fue 39,01; con *stall*=10 el algoritmo corrió, en promedio, 65,2 iteraciones antes de detenerse por sí solo y llegó a una media

ligeramente mejor (38,90); con un presupuesto de 5 s alcanzó apenas 38 iteraciones en promedio y quedó peor (39,31). Es decir, para esta instancia, dejar que el algoritmo decida cuándo parar (*stall*) fue tan bueno como fijar un número de iteraciones, y mejor que limitar el tiempo de forma estricta; las corridas de este informe usan, de todas formas, un número de iteraciones fijo (más fácil de comparar entre configuraciones distintas).

Métricas y su medición. Longitud del mejor tour (recalculada en doble precisión y verificada contra la reportada por el programa en cada corrida), tiempo de preparación y de ACO (`std::chrono::steady_clock`), y pico de memoria del proceso (`GetProcessMemoryInfo/PeakWorkingSetSize` en Windows). Para comparar configuraciones se usa la media, la desviación estándar, el mínimo y el máximo sobre las semillas, y una prueba de Wilcoxon apareada (mismas semillas en ambas configuraciones) antes de afirmar que una es mejor que otra, no solo la diferencia de medias.

6 Resultados por instancia

Todas las tablas de esta sección se generan automáticamente con `scripts/generar_figuras.py` a partir de los CSV de `resultados/`; ningún número se copió a mano.

6.1 $n = 20$

5 semillas, $m = 20$, $K = 19$, contra el óptimo exacto de Held-Karp.

Tabla 6: $n=20$ contra Held-Karp, $m=20$, $K=19$, 5 semillas.

Semilla	L_{mejor}	Óptimo exacto	Gap (%)	Tour válido
1	4,154	4,106	1,176	sí
2	4,270	4,270	0,000	sí
3	3,537	3,537	0,000	sí
4	4,075	4,075	0,000	sí
5	4,266	4,266	0,000	sí

Media 0,235, desviación 0,526, mínimo 0,000, máximo 1,176 (puntos porcentuales de gap).

Cuatro de cinco semillas alcanzan el óptimo exacto; la semilla restante queda a 1,176 % del óptimo. Los cinco tours son permutaciones válidas.

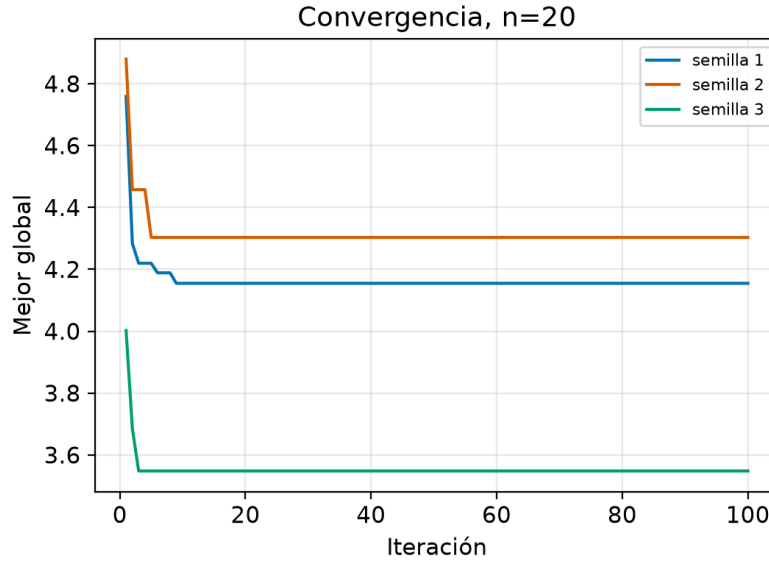


Figura 2: Convergencia (mejor global vs. iteración), $n=20$, 3 semillas.

Cómo leer esta figura: cada línea es el mejor tour encontrado hasta esa iteración, para una semilla distinta. Se espera una caída rápida en las primeras iteraciones (la feromona todavía no distingue buenas de malas aristas) y un aplanamiento posterior (una vez que la feromona converge alrededor de las aristas del mejor tour, hay cada vez menos margen de mejora). Las tres semillas convergen a valores parecidos, coherente con que $n = 20$ es una instancia pequeña donde el algoritmo explora casi todo el espacio relevante en pocas iteraciones.

6.2 $n = 2\ 000$

8 semillas, configuración ganadora ($K = 8$, $\alpha = 1.5$, $\beta = 5$, $\rho = 0.1$, $q_{fac} = 3$, segunda feromona activada), $m = 2\ 000$, 50 iteraciones.

Tabla 7: $n=2\ 000$, configuración ganadora, $m=2\ 000$, $K=8$, 8 semillas.

Media L_{mejor}	Desv. estándar	Mínimo	Máximo
37,496	0,352	36,939	38,005

La nota preliminar del enunciado (otra máquina, una sola semilla, configuración sin segunda feromona) reportaba una longitud de 38,20 para una configuración parecida; el resultado con 8 semillas y la configuración ganadora es consistente en orden de magnitud y ligeramente mejor.

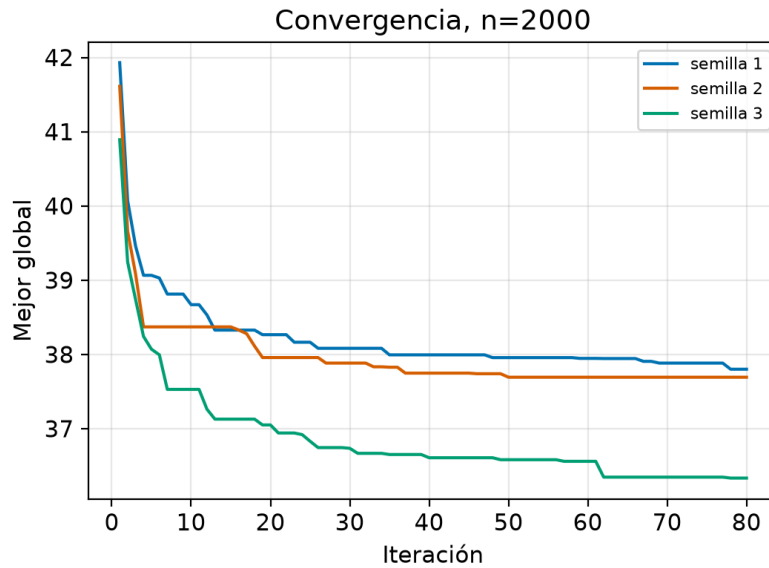


Figura 3: Convergencia (mejor global vs. iteración), n=2 000, 3 semillas.

Cómo leer esta figura: igual que en $n = 20$, pero con una caída más gradual: al haber más ciudades y más pasos por tour, cada iteración aporta información más fina, y la mejora se reparte en más iteraciones antes de aplanarse.

Antes de leer los mapas de calor de α - β y ρ - Q que siguen, conviene recordar qué controla cada parámetro de la ecuación (2) y de la actualización de feromona (sección 3.4): α , β y ρ regulan el equilibrio entre repetir lo que ya funcionó (**explotación**) y probar caminos distintos (**exploración**). α alto amplifica la feromona acumulada y favorece la explotación (riesgo: estancamiento prematuro); β alto favorece los vecinos cercanos sin importar la feromona (riesgo: ignorar información útil de iteraciones pasadas); ρ alto olvida rápido el pasado (más exploración, pero también más varianza entre corridas), equilibrio que barren, precisamente, los dos mapas de calor de abajo en esta instancia:

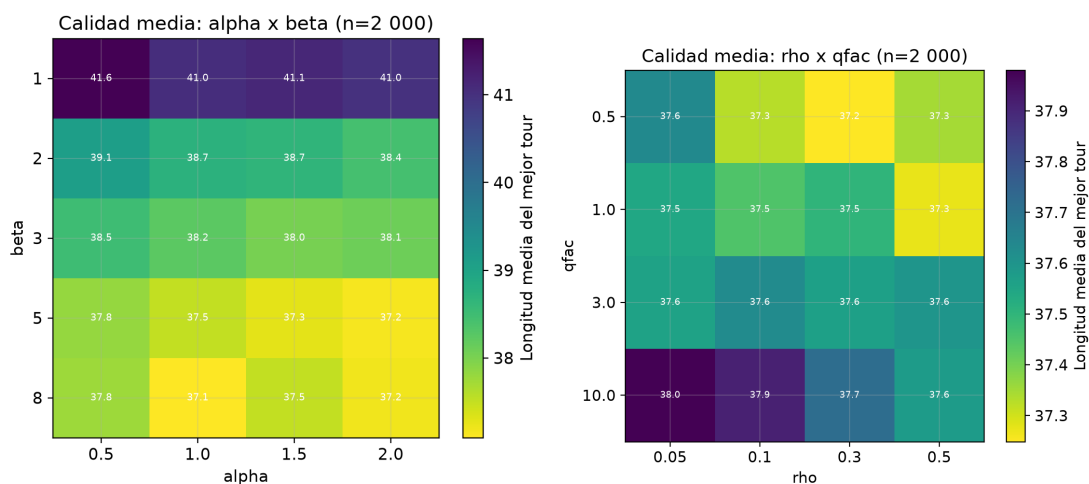


Figura 4: Superficie de calidad media, n=2 000, 3 semillas por celda: $\alpha \times \beta$ (izquierda) y $\rho \times qfac$ (derecha).

Cómo leer esta figura: cada celda es la longitud media del mejor tour para esa combinación de parámetros (más oscuro/morado es peor, más claro/amarillo es mejor). El mapa alpha-beta muestra que **beta** domina: la calidad mejora de forma consistente de arriba (beta=1) hacia abajo (beta=8), casi sin importar la columna (alpha), mientras que alpha por sí solo cambia poco la calidad dentro de cada fila. Esto es coherente con la teoría: beta pesa la heurística (distancia), que en un TSP euclidiano es información muy confiable desde la primera iteración, mientras que alpha pesa la feromona, que recién se vuelve informativa después de varias iteraciones. El mapa rho-qfac muestra una superficie mucho más plana (rango de apenas 37,2 a 38,0): ninguno de los dos domina con la fuerza de beta, y la mejor zona (qfac bajo a moderado, rho moderado a alto) es ancha, no un único punto aislado — el algoritmo, en esta instancia, es razonablemente robusto a estos dos parámetros en concreto (observación de este trabajo, no una cita de la literatura).

6.3 $n = 200\,000$

Configuración ganadora, 3 semillas, dos corridas con el mismo presupuesto de tiempo de reloj (~15 min/semilla) pero distinto número de hormigas (detalle y análisis en la sección 7 y en docs/decisiones_diseno.md):

- $m = 20000$ (recomendación de la literatura, 10 iteraciones): L_{mejor} media 387,308 (desv. 0,190), mejora media del 1,28 % sobre el vecino más cercano, pico de memoria 102,4 MB.
- $m = 2048$ (igual presupuesto de tiempo, ≈ 108 iteraciones): L_{mejor} media 385,485 (desv. 0,550), mejora media del 1,75 % sobre el vecino más cercano, pico de memoria 102,4 MB.

Los cinco tours de ambas corridas se verificaron como permutaciones válidas. El hallazgo más relevante de esta instancia no es la calidad absoluta (ambas configuraciones quedan lejos del óptimo, esperable con tan pocas iteraciones sobre 200 000 ciudades) sino la comparación entre ellas: ver sección 7.

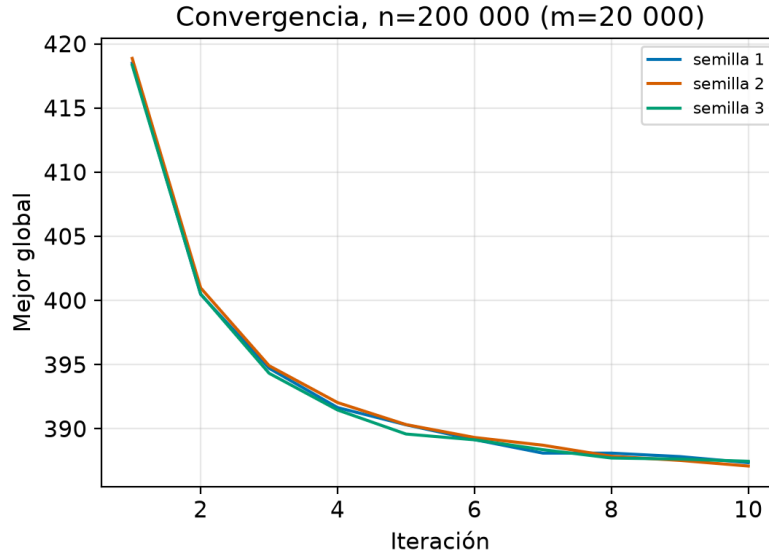


Figura 5: Convergencia (mejor global vs. iteración), $n=200\,000$, $m=20\,000$, 3 semillas.

Cómo leer esta figura: solo 10 iteraciones, muy pocas frente a las 50-100 usadas en $n = 20$ y $n = 2000$; la curva todavía cae con pendiente notable en la última iteración graficada, es decir, el algoritmo no llegó a aplanarse — coherente con la discusión de la sección 7 sobre qué tan lejos se puede llegar con el presupuesto de tiempo de esta máquina.

7 Análisis de tiempo y memoria

7.1 Memoria: densa contra dispersa

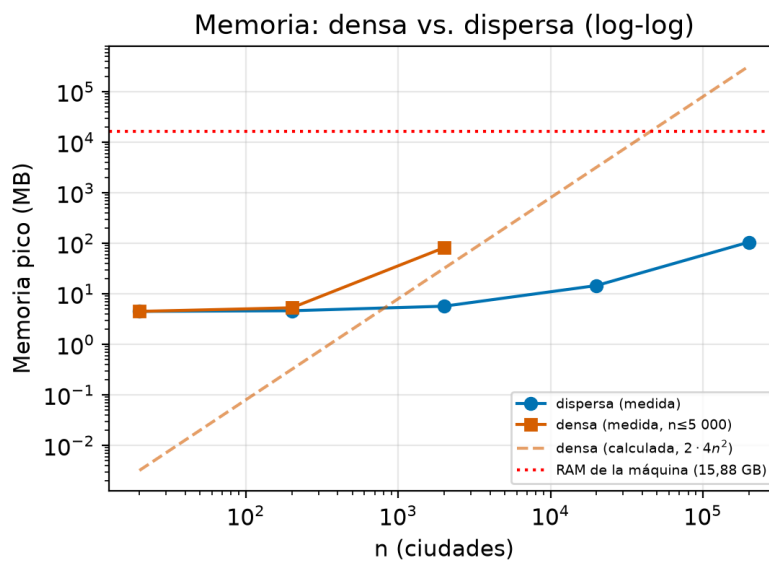


Figura 6: Memoria pico contra n , versión densa contra dispersa, escala log-log.

Cómo leer esta figura: la línea densa (medida hasta $n = 2000$, calculada más allá) crece como n^2 y cruza la RAM disponible de la máquina (15,88 GB) alrededor de $n \approx 63000$; la línea dispersa crece de forma mucho más lenta ($O(nK)$) y sigue siendo de decenas de MB incluso en $n = 200000$. Esta es la comprobación empírica directa de por qué el enunciado pide “ahorrar recursos”: sin la lista de K candidatas, la instancia más grande de este trabajo sería, sencillamente, irrealizable en esta máquina.

7.2 Memoria por componente

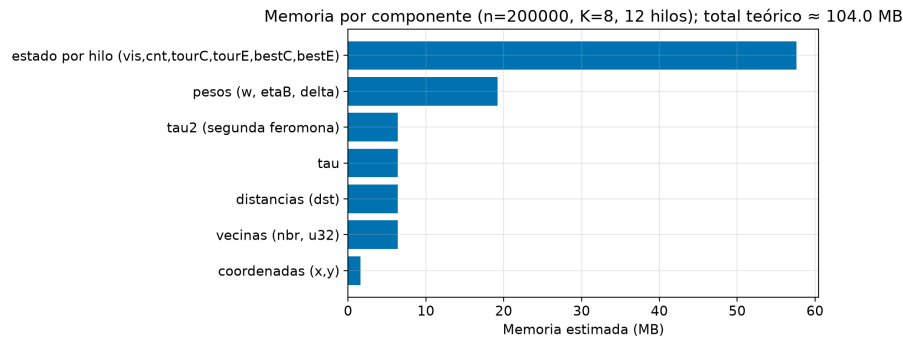


Figura 7: Memoria estimada por componente, $n=200000$, $K=8$, 12 hilos.

Cómo leer esta figura: cada barra es una estimación teórica directa a partir de los tamaños de los arreglos del código (coordenadas $2n$, vecinas y distancias nK cada una, feromona y feromona secundaria nK cada una, pesos $3nK$, y el estado por hilo $6n$ por cada uno de los 12 hilos). La suma teórica (104,0 MB) coincide, dentro de un 1,3 %, con el pico de memoria realmente medido en la misma instancia (105,4 MB, resultados/escalamiento_disperso.csv): esa cercanía confirma que el modelo de memoria de la sección 4 explica correctamente dónde se va cada byte, no solo el orden de magnitud total.

7.3 Efecto de K y de la segunda feromona

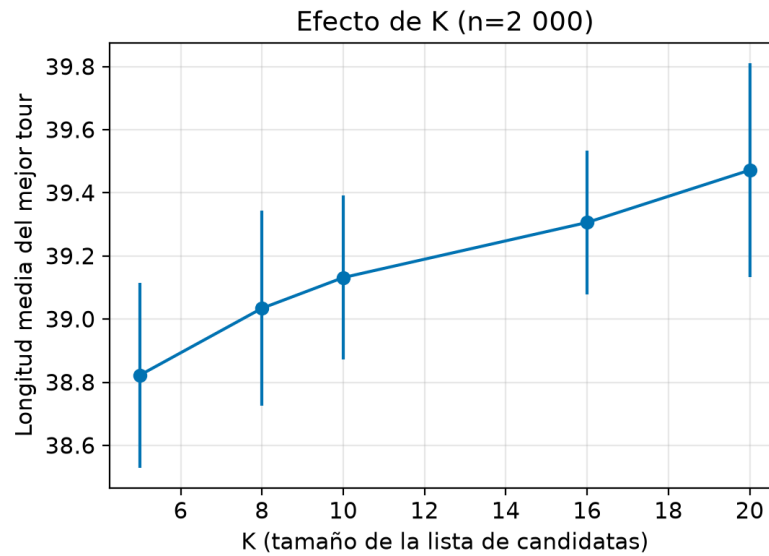


Figura 8: Efecto de K sobre la calidad, $n=2\,000$ (media $\pm$ desviación, 5 semillas).

Cómo leer esta figura: a diferencia de lo que podría esperarse (más candidatas, mejor calidad), la media empeora ligeramente al aumentar K en esta configuración base. Con $\alpha = 1$, $\beta = 3$ (valores de esta barrida, no los de la configuración ganadora), más candidatas dan a la ruleta más opciones de peso parecido entre sí, lo que diluye la guía de la heurística; esto es una observación empírica de esta configuración concreta, no una regla general sobre K .

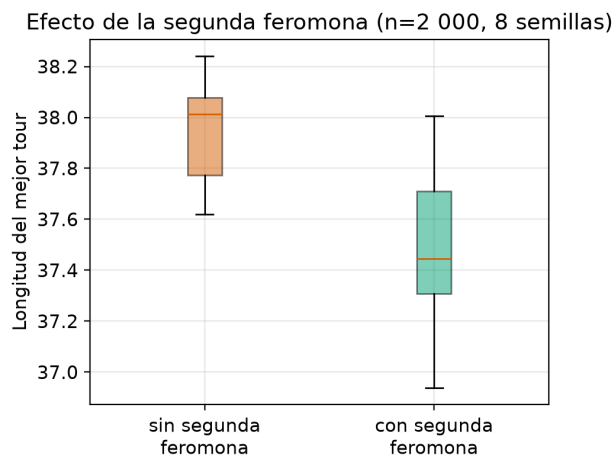


Figura 9: Efecto de la segunda feromona sobre la configuración de referencia, $n=2\,000$, 8 semillas.

Cómo leer esta figura: misma comparación apareada de la sección 5, mostrada como diagrama de caja: la caja “con segunda feromona” está desplazada hacia longitudes

menores, con una dispersión similar a la de la configuración sin ella, es decir, la mejora no viene a costa de más varianza entre semillas.

7.4 Varianza entre semillas por parámetro

Los diagramas de caja de cada parámetro del barrido (`figuras/03_boxplot_*.png`) muestran, para cada valor probado, la dispersión de la longitud del mejor tour entre las 5 semillas. Se incluyen en el anexo de figuras; ninguno muestra una semilla atípica que se aleje de forma anómala de las demás, lo que respalda usar la media (y no solo una corrida) como criterio de comparación entre valores de un parámetro.

7.5 Longitud contra referencias

Tabla 8: Longitud del mejor tour contra óptimo exacto ($n=20$) y aproximación BHH (n grande).

Instancia	L_{ACO}	Óptimo exacto	Aprox. BHH
$n=20$	4.060	4.051	–
$n=2\,000$	37.496	–	32.911
$n=200\,000$	387.308	–	329.109

Para $n = 2000$, la longitud del mejor tour (37,50) queda por encima de la aproximación asintótica de Beardwood-Halton-Hammersley (32,91), como se espera: BHH es un límite asintótico para $n \rightarrow \infty$ en el cuadrado unidad, no una cota alcanzable en una instancia finita, y además supone puntos uniformes sin borde, mientras que el cuadrado unidad sí tiene borde (lo que sesga hacia tours algo más largos que la aproximación). Para $n = 20$, la comparación relevante es Held-Karp (óptimo exacto), no BHH (que solo es significativo para n grande).

7.6 Costo de m y escalamiento con n

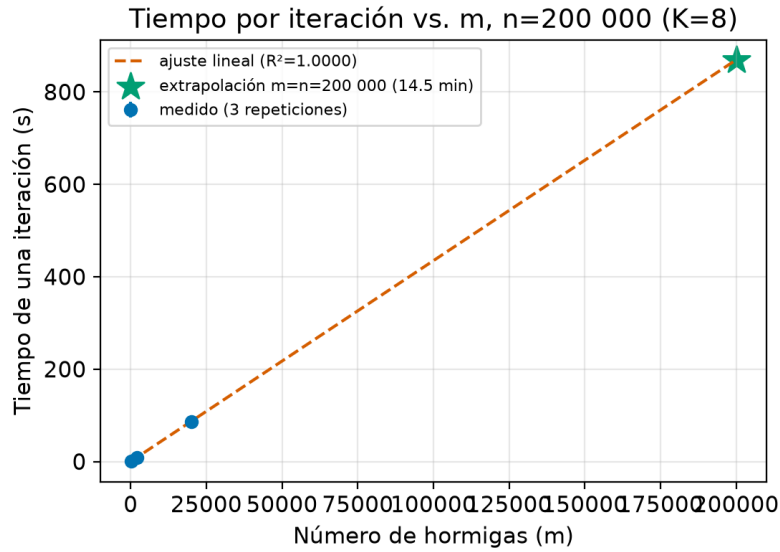


Figura 10: Tiempo de una iteración vs. m , $n=200\,000$, $K=8$, con ajuste lineal y extrapolación.

Tabla 9: Tiempo de una iteración vs. m , $n=200\,000$, $K=8$ (3 repeticiones).

m	Tiempo medio (s)	Desv. estándar (s)
200	0,850	0,015
2000	8,681	0,066
20000	86,940	0,476

Ajuste: $t(m) \approx 0,004348m - 0,017$ s, $R^2 = 1,000000$. Extrapolación a $m = n = 200\,000$: 869,6 s ($\approx 14,5$ min), **estimación por extrapolación, no corrida real.**

Cómo leer esta figura: los tres puntos medidos (con barras de error de 3 repeticiones) caen casi exactamente sobre la recta ajustada ($R^2 = 1,000000$), confirmando que el costo de una iteración es lineal en m a n y K fijos, tal como predice la derivación $O(mnK)$ (sección 4). La estrella marca la extrapolación a $m = n = 200\,000$: está fuera del rango medido, por eso se marca explícitamente como estimación y no como dato.

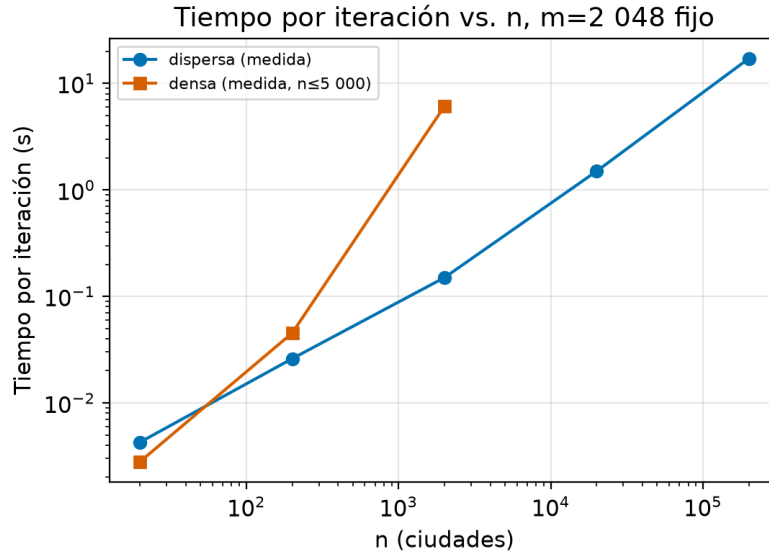


Figura 11: Tiempo por iteración vs. n , $m=2\,048$ fijo, dispersa vs. densa, log-log.

Cómo leer esta figura: en escala log-log, una relación de potencia $t \propto n^p$ se ve como una línea recta de pendiente p . La versión densa crece más rápido (mayor pendiente, consistente con $O(n^2)$ en tiempo más el costo de construir la matriz) que la dispersa ($O(nK)$, pendiente cercana a 1). Esta es la comprobación empírica, en tiempo, del mismo argumento que la Figura 6 hace en memoria.

7.7 Igual presupuesto de cómputo en $n = 200\,000$

Ver la tabla 2 y la discusión de la sección 3.3: con el mismo tiempo de reloj, $m = 2048$ (más iteraciones) superó a $m = 20\,000$ (menos iteraciones) en las tres semillas probadas.

7.8 Tours finales

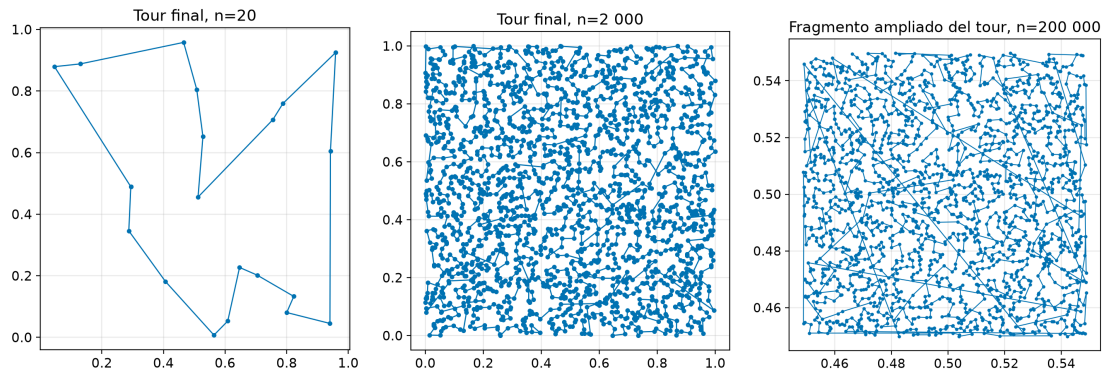


Figura 12: Tour final de $n=20$ y $n=2\,000$, y un fragmento ampliado (recorte de $\pm 0,05$ alrededor del centro) del tour de $n=200\,000$.

Cómo leer esta figura: verificación visual de que el tour es coherente (sin cruces evidentes ni saltos absurdos) para $n = 20$ y $n = 2000$. El fragmento de $n = 200000$ corresponde a una corrida corta e ilustrativa (11 iteraciones, no la corrida oficial de la sección 7), y muestra líneas largas que cruzan el recorte: con pocas iteraciones sobre una instancia tan densa, el tour todavía no termina de “localizarse” dentro de cada vecindario denso de ciudades, algo consistente con la limitación de calidad-con-pocas-iteraciones discutida en la sección de limitaciones.

8 Limitaciones

- **ACO no garantiza el óptimo.** Es una metaheurística: encuentra buenas soluciones en tiempo razonable, no la mejor solución garantizada. La única instancia con verificación exacta es $n = 20$ (Held-Karp).
- **Con $n = 200000$ y pocas iteraciones, la mejora sobre el vecino más cercano es pequeña.** El costo de una iteración crece de forma lineal en m (sección 3.3), y la feromona se actualiza una sola vez por iteración sin importar cuántas hormigas corrieron esa iteración: con un presupuesto de tiempo fijo, hay una tensión directa entre correr más hormigas por iteración o más iteraciones, y ambas cuestan tiempo de la misma bolsa.
- **La lista de K candidatas restringe el espacio de búsqueda.** Una hormiga solo elige, en el caso habitual, entre las K vecinas más cercanas de la ciudad actual; esto acelera la construcción pero excluye, por diseño, aristas hacia ciudades lejanas que en principio podrían ser parte del tour óptimo (poco probable en un TSP euclidiano, pero no imposible).
- **Cualquier extrapolación se marca como tal.** El costo de una iteración con $m = n = 200000$ (sección 3.3) es una extrapolación por mínimos cuadrados sobre solo tres puntos medidos, no una corrida real; se reporta junto con su coeficiente de determinación para que el lector juzgue la confianza de esa cifra.

9 Conclusiones

1. El espacio de búsqueda del TSP crece de forma factorial (sección 2); para $n = 200000$ ninguna búsqueda exhaustiva es concebible, y el aporte central de este trabajo es una representación de datos ($O(nK)$, no $O(n^2)$) que hace viable siquiera intentar esa instancia en una computadora personal.
2. Ahorrar memoria y ahorrar tiempo, en este problema, son la misma decisión de diseño: la lista de K vecinas, la rejilla espacial y los buffers por hilo (sección 3.4)

resuelven ambos a la vez, no por separado.

3. La convención de la literatura ($m = n$ para Ant System) tiene respaldo teórico verificado (sección 3.3), pero en la práctica de este equipo, con esta máquina y este tiempo disponible, no es ejecutable para $n = 200\,000$; el tope adoptado ($m = 20\,000$) es una decisión de ingeniería explícita, no una regla universal. La comparación directa a igual presupuesto de tiempo en $n = 200\,000$ (sección 7) incluso sugiere lo contrario de la intuición de “más hormigas es mejor”: con el mismo tiempo de reloj, más iteraciones con menos hormigas ($m = 2\,048$) dieron mejor calidad que menos iteraciones con más hormigas ($m = 20\,000$), en las tres semillas probadas. Ese ahorro tiene, sin embargo, un límite razonable: reducir agentes e iteraciones por debajo de cierto punto sí ahorra tiempo de forma proporcional, pero degrada notablemente la calidad de la solución, así que las dos configuraciones “buenas” documentadas ($m = 20\,000 \times 10$ y $m = 2\,048 \times 108$) representan puntos más equilibrados de este compromiso que uno de recorte extremo.
4. Optimizar cada parámetro por separado no equivale a optimizar la combinación conjunta: el barrido de un factor a la vez identificó una mejora real (la segunda capa de feromona), pero también un espejismo (combinar los mejores valores individuales no mejoró el resultado).
5. La elección de C++ con OpenMP fue una decisión práctica (código base validado, paralelismo casi sin código adicional), no el resultado de descartar otros lenguajes por incapacidad: C y Rust habrían sido, en principio, igual de aptos.

Ver `docs/RESUMEN_FINAL.md` para el resumen ejecutivo de todo el proyecto, incluidas las corridas del bloque de cierre.